

bomblab 报告

姓名：罗翌宁

学号：2024201573

总分	phase_1	phase_2	phase_3	phase_4	phase_5	phase_6	secret_phase
7	1	1	1	1	1	1	1

scoreboard 截图：

2024201573	0	0	0	0	0	1	0	0	0	0	0	0	0	1	7
------------	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

解题报告

phase_1

A fading Light awaits two in a closed world of forgotten Conflict.

讲解题目思路

通过读汇编我们不难发现，这个题目对应的代码大致如下：

```
void phase_1(){
    string s;
    getline(cin,s);
    if(s!=_s){//这里的_s是一个已经存储在0x1d40+%rip的字符串
        explode_bomb();
    }
}
```

这里进行的是将0x1d40+%rip传入%rsi，并传入strings_not_equal与我们输入的字符串进行对比，所以我们只需要找到0x1d40+%rip处存储的字符串就可以了

运行时rip会指向 1440: call strings_not_equal，在Layout里面可以看到这一行的地址是0x05555555440

所以最后传入strings_not_equal的地址就是0x55555555440+0x1d40=0x555555557180

所以我们直接在运行时用x/s0x555555557180这个地址存储的字符串就可以找到答案

phase_2

1256597 524853 1248567 411412

首先来看一下这个题目要求的输入：

```

1464:  48 89 44 24 28      mov    %rax,0x28(%rsp)
1469:  31 c0                xor     %eax,%eax
146b:  48 89 e2             mov     %rsp,%rdx
146e:  48 8d 4c 24 04       lea     0x4(%rsp),%rcx
1473:  4c 8d 4c 24 0c       lea     0xc(%rsp),%r9
1478:  4c 8d 44 24 08       lea     0x8(%rsp),%r8
147d:  48 8d 35 6d 21 00 00 lea     0x216d(%rip),%rsi      # 35f1
<array.0+0x391>
1484:  e8 c7 fc ff ff      call   1150 <__isoc99_sscanf@plt>
1489:  83 f8 04             cmp     $0x4,%eax

```

这里调用了 `__isoc99_sscanf@plt`，上面的上面的4行说明我们要把函数的参数分别输入到 `%rsp`，`0x4(%rsp)`，`0x8(%rsp)`，`0xc(%rsp)`，而下面的判断 `sscanf` 函数返回值是否等于4也说明了我们理论上应该需要输入4个整数

下面我们来寻找输入的4个整数应该是多少

```

<phase_2+0x4d>
148e:  48 8d 3d ab 4c 00 00 lea     0x4cab(%rip),%rdi # 6140 <matA.3>
1495:  48 8d 5c 24 10       lea     0x10(%rsp),%rbx
149a:  41 bb 00 00 00 00     mov     $0x0,%r11d
14a0:  eb 19               jmp     14bb

```

这段代码将 `0x4cab+%rip` 存入了 `rdi`，通过注释我们知道这里应该是指向了一个 `matA` 矩阵的起始地址。 `rbx = rsp + 0x10` 可间是为存放最后的输出结果预留出了一段栈上的空间。

而后就进入了我们整个代码的主题部分，即计算对应的四个数字的主题循环部分。

```

14a9:  41 83 c3 01          add     $0x1,%r11d
14ad:  48 83 c7 0c          add     $0xc,%rdi
14b1:  48 83 c3 08          add     $0x8,%rbx
14b5:  41 83 fb 02          cmp     $0x2,%r11d

```

这里对应的是循环的外层，每次循环对 `%r11d + 1`，对 `%rdi + 0xc`，我们之前已经知道 `rdi` 存储的是 `matA`，所以就是每次对 `matA` 往右移动12个字节也就是对 `matA` 的索引 + 3； `%rbx + 8` 则是说明会往后移动8个字节，也就是2个 `int`，也就是说明一次循环内要计算2个 `int`，最后的 `cmp` 告诉我们这个循环要进行两次

接下来进行内层循环，内层循环用 `%r8d` 计数，更内层的用了一个 `rax` 来记录在数组中计算到第几个了

```

14d8:  8b 14 87             mov     (%rdi,%rax,4),%edx
14db:  0f af 14 c6          imul    (%rsi,%rax,8),%edx
14df:  01 d1               add     %edx,%ecx

```

这里记录了我们最后对应的每个数字的计算的法则 $sum1 += matA[i+j] * matB32[j2]$, $sum2 += matA[i+j] * matB32[j2 + 1]$, i 表示起始位置, 在 $sum1$ 计算的时候不会变, j 存储在 eax 中由0变到3。所以我们复原出来大致的代码如下:

```

outputs = []
int rdi_index = 0;
for (outer=0;outer<2;outer++){
    rsi_byte_offset = 0
    for (int inner=0;inner<2;inner++){
        s = 0
        for (int k=0;k<4;k++){
            a = A[rdi_index + k]
            b = matB_base[rsi_byte_offset + k*8]
            s += a * b
        }
        outputs.push_back(s)
        rsi_byte_offset += 4
    }
    rdi_index += 3
}

```

然后我们就只需要用

```

(gdb) x/16gx 0x55555555140
(gdb) x/16gx 0x55555555120

```

分别查看 $matA$ 和 $matB$ 中的存储的数就可以了。

$matA = [222, 869, 761, 589, 767, 185, 0, 0, 256, 0, 0, 0, 856, 6, 0, 0]$

$matB_{32} = [886, 169, 844, 348, 429, 243, 0, 0]$

$sum1 = 222*886 + 869*844 + 761*429 + 589*0 = 1256597$

$sum2 = 222*2704 + 869*348 + 761*3839 + 589*0 = 524853$

$sum3 = 589*886 + 767*884 + 185*429 + 0*0 = 1248567$

$sum4 = 589*2704 + 767*348 + 185*3839 + 0*0 = 411412$

四个密码就破译完了

phase_3

0 D 761

1 E 396

```

2 A 883
3 O 600
4 S 567
5 I 364
6 B 157
7 Q 238

```

首先我们分析这个题目要求我们进行的输入应该是什么：

```

1558:  48 8d 4c 24 0f      lea    0xf(%rsp),%rcx
155d:  48 8d 54 24 10      lea    0x10(%rsp),%rdx
1562:  4c 8d 44 24 14      lea    0x14(%rsp),%r8
1567:  48 8d 35 8f 1c 00 00 lea    0x1c8f(%rip),%rsi      # 31fd
<_IO_stdin_used+0x1fd>
156e:  e8 dd fb ff ff      call   1150 <__isoc99_sscanf@plt>
1573:  83 f8 02            cmp     $0x2,%eax

```

从这里我们可以分析得到我们依次需要输入int,char,int到%rdx, %rcx和%r8, 输入错误没有成功输入3个就会爆炸

```

1578:  8b 05 92 4b 00 00    mov     0x4b92(%rip),%eax      # 6110
<mask.1>
157e:  30 44 24 0f         xor     %al,0xf(%rsp)

```

这里我们可以看到是对0xf(%rsp)用掩码%al进行了xor操作, 通过用p/x (*unsigned char*)查看我们发现%al存储的是0x20, 对char做与0x20异或就相当于大小写的转换, 所以这里0xf(%rsp)存储的char发生了大小写转化。

```

1582:  83 7c 24 10 07      cmpl    $0x7,0x10(%rsp)
1587:  0f 87 0c 01 00 00    ja      1699 <phase_3+0x155>

```

从这里我们得知输入0x10(%rsp)的整数应该小于等于7, 否则也会引爆炸弹

```

158d:  8b 44 24 10         mov     0x10(%rsp),%eax
1591:  48 8d 15 a8 1c 00 00 lea     0x1ca8(%rip),%rdx      # 3240
<_IO_stdin_used+0x240>
1598:  48 63 04 82         movslq  (%rdx,%rax,4),%rax
159c:  48 01 d0            add     %rdx,%rax
159f:  ff e0              jmp     *%rax

```

这一部分很明显是switch的跳转部分，跳转的地址最终被存储在%rax中，通过查看我们我们会发现0x10(%rsp)中存储的是i时候，分别会跳转到0x5555555a8+0x22*i

以0x10(%rsp)存的是0的情况为例：

```
15a8:  b8 64 00 00 00      mov    $0x64,%eax
15ad:  81 7c 24 14 f9 02 00  cmpl   $0x2f9,0x14(%rsp)
```

说明此时输入0x14(%rsp)的应该是\$0x2f9即761，相等才会跳转到16a3，否则会引爆炸弹

```
16a3:  38 44 24 0f          cmp    %al,0xf(%rsp)
16a7:  75 15                jne    16be <phase_3+0x17a>
```

跳转到16a3后会把%al和0xf(%rsp)进行比较，case0下%al=64，即d，而由于0xf(%rsp)已经发生过大小写的转化，所以最开始输入的是D

以此同样的类推，我们可以发现phase3 的8组密码： 0 D 761； 1 E 396； 2 A 883； 3 O 600； 4 S 567； 5 I 364； 6 B 157； 7 Q 238

phase_4

31BA

```
17a3:  48 8d 54 24 0c      lea    0xc(%rsp),%rdx      # 12+rsp
17a8:  48 8d 35 57 1a 00 00 lea    0x1a57(%rip),%rsi   # 3206
<_IO_stdin_used+0x206>
17af:  e8 9c f9 ff ff      call   1150 <__isoc99_sscanf@plt>
17b4:  83 f8 02            cmp    $0x2,%eax
```

从这里我们可以看出phase_4要求我们输入2个，结合下面的cmp和strings_not_equal的调用，我们可以知晓输入一个整数一个字符串

函数首先会调用func_1,通过对func_1的代码进行分析我们可以很轻松地知晓他是一个递归函数，大致源代码如下：

```
int func_1(int n){
    if (n==1){return 1;}
    else{return 2*func_1(n-1)+1;}
}
```

而在此处我们输入的参数n=%edi=5； func_1(5)=31,所以第一个要输入的数字就是31。

```

17ce:  e8 23 05 00 00      call 1cf6 <string_length>
17d3:  83 f8 02           cmp $0x2,%eax

```

由此处我们可以得知我们要输入的字符串的长度应该是2

接下来函数调用func_2，输入func_2的参数r8d = 0x42，ecx = 0x43，edx = 0x41，esi = 0x14，edi = 0x5，最后返回的结果在%r9里

对func_2分析后我们得到他大致的源码：

```

void func4_2(int n, int k, char A, char C, char B, char* buf) {
    if (n == 1) {
        buf[0] = A;
        buf[1] = C;
        buf[2] = '\0';
        return;
    }
    int mid = func4_1(n - 1);
    if (k <= mid) {
        func4_2(n - 1, k, A, B, C, buf);
    } else if (k == mid + 1) {
        buf[0] = A;
        buf[1] = C;
        buf[2] = '\0';
    } else {
        func4_2(n - 1, k - mid - 1, B, C, A, buf);
    }
}

```

所以我们可以知道，初始fun4_2(5,20,A,C,B) 进入target > mid+1 → fun4_2(4,4,B,C,A) 进入target <= mid → fun4_2(3,4,B,A,C) 进入target = mid+1 → 输出BA

所以最后的输出是BA

所以最后的密码就是31BA

phase_5

```
456:<>
```

```

184b:  e8 a6 04 00 00      call 1cf6 <string_length>
1850:  83 f8 06           cmp $0x6,%eax

```

从这里我们知道phase_5的输入应该是长度为6的字符串

分析可知, ecx的值大致如下变化:

7 / 11

之后我们进入到这个题目最关键的部分，即根据a[i]存储的值对链表进行重排

通过分析代码，我们不难发现以0x48f2(%rip)为起始地址的链表的一个Node的大小为16个Byte，后8个Byte存储的是下一个节点的地址，即大致是下面这种结构：

```
struct node{
    int value;
    node* next;
}
```

查看内存我们发现，初始时候6个node的值分别是：

node1	0x00000000000006220	816 (0x330)	0x0005555555555a2a	(node2)
node2	0x0005555555555a2a	336 (0x150)	0x0005555555555a2a	(node3)
node3	0x0005555555555a2a	272 (0x103)	0x0005555555555a2a	(node4)
node4	0x0005555555555a2a	813 (0x32d)	0x0005555555555a2a	(node5)
node5	0x0005555555555a2a	684 (0x2ac)	0x0005555555555a2a	(node6)
node6	0x0005555555555a2a	856 (0x358)	0x0000000000000000	(NULL)

而在链表重组这里，我们会先按照改变后a[i]存储的数字的顺序，将node按照此顺序存入数组0x30(%rsp,%rsi,8)中，然后再按照这个里面的顺序，将node_a[i]的next设置为&node_a[i+1]，实现了数组的重新串联

接下来会判断新链接得到的链表是否单调递减，否则将引爆炸弹，单调递减则拆除成功

将node排序，6>1>4>5>2>3

所以我们输入的应该是：1 6 3 2 5 4

至于enigma，是为了我们能够成功进入secret_phase，将在下一问分析

secret_phase

```
ccaac
```

入口的寻找

首先我们需要寻找secret_phase的入口，我运用ctrl+f检索secret_phase，成功找到入口在<phase_defused>中，也就是说在某一个phase拆除后，可以传入一个额外的信息以启动secret_phase，我们来看启动条件

```
21c1: 83 3d 70 45 00 00 06    cmp1    $0x6,0x4570(%rip)          # 6738
<num_input_strings>
21c8: 74 05                    je      21cf <phase_defused+0x1c>
```


这里要求 $0x4570(\%rip) == 6$ 才会进入判断程序，经过检测这个变量我们发现他应该是记录我们已经输入了多少次了的一个变量，所以就是在phase_6之后才会进入这个判断程序。

后面会对输入的字符串里面的空格进行计数，只有当空格数等于6的时候才会继续往下走，也就是说，我们需要在phase_6的答案后面加个空格，而后他会对比 $(\%rsi, \%rax, 1)$ 和 $0x1404(\%rip)$ 中的string是否相等，相等就可以进入secret_phase。 $(\%rsi, \%rax, 1)$ 就是我们输入的空格后一个位置，通过读取内存我们发现 $0x1404(\%rip)$ 中存储的是enigma，所以我们只需要在phase_6的答案后加上" enigma"就可以进入secret_phase。

密码的寻找

很显然这里需要我们输入一段长度小于20的字符串然后传入func7。func7是一个很有趣的类似yoy上面一道马走日的题目，我们输入的字符串会控制马的移动，到达终点就能成功拆弹。

传入函数的内容如下： $\%rdi$ = 字符串指针; $\%esi$ = 当前 X 坐标; $\%edx$ = 当前 Y 坐标; $\%ecx$ = 当前字符索引。

之后一大段的move都是和我们字符——对应的(dx,dy)的移动操作，在 $\%rsp - \%rsp + 19$ 对应的是x的移动， $\%rsp + 20 - \%rsp + 39$ 对应y的移动。

```
1b16:  83 fe 04      cmp     $0x4,%esi
1b19:  75 6b         jne     1b86 <func7+0x18e>
1b1b:  83 fa 07      cmp     $0x7,%edx
1b1e:  75 66         jne     1b86 <func7+0x18e>
```

这里告诉了我们最后胜利的终点是 $\%esi=4$ ， $\%edx=\$0x7$ ；当指向字符串末尾的时候，会自动进入最后的是否走到重点的判断。

否则会进入以下程序控制马的移动：

```
1b3c:  41 89 f2      mov     %esi,%r10d
1b3f:  41 83 e2 07    and     $0x7,%r10d
1b43:  83 e6 07      and     $0x7,%esi
1b46:  41 89 c0      mov     %eax,%r8d
1b49:  44 03 04 b4    add     (%rsp,%rsi,4),%r8d
1b4d:  41 89 d3      mov     %edx,%r11d
1b50:  44 03 5c b4 20 add     0x20(%rsp,%rsi,4),%r11d
```

将目前指向的字符串中的字符提取出后3位用于控制移动。比如 $a \& 0x7 = 1$ ，此时a对应的dx就是 $rsp + 4 * 1$ 即-1，dy就是 $rsp + 4 * 1 + 20$ 即2。根据次规则，我们可以得到以下的字符对应的移动分别是：
a(-1,2);b(1,2);c(2,1);d(2,-1).....

```
1b58:  44 09 de      or      %r11d,%esi
1b5b:  b9 00 00 00 00 mov     $0x0,%ecx
1b60:  83 fe 07      cmp     $0x7,%esi
1b63:  76 3f         jbe     1ba4 <func7+0x1ac>
```

这里则是对x和y进行越界检测，保证x和y在0-7的范围内。

```

1ba7: 42 03 44 94 40      add    0x40(%rsp,%r10,4),%eax    # x的蹩马脚
1bac: 42 03 54 94 60      add    0x60(%rsp,%r10,4),%edx    # y的蹩马脚
1bb1: 48 8d 35 f8 45 00 00  lea     0x45f8(%rip),%rsi        # 61b0 <row0>
1bb1: 48 8d 35 f8 45 00 00  lea     0x45f8(%rip),%rsi        # 61b0 <row0>
1bb8: 85 c0               test    %eax,%eax
1bba: 7e 0b               jle     1bc7 <func7+0x1cf>
1bbc: 48 8b 76 08         mov     0x8(%rsi),%rsi
1bc0: 83 c1 01            add     $0x1,%ecx
1bc3: 39 c8               cmp     %ecx,%eax
1bc5: 75 f5               jne     1bbc <func7+0x1c4>
1bc7: 48 63 d2            movslq  %edx,%rdx
1bca: b9 00 00 00 00      mov     $0x0,%ecx
1bcf: 80 3c 16 01         cmpb    $0x1,(%rsi,%rdx,1)
1bd3: 74 90               je      1b65 <func7+0x16d>

```

这里引入了一个非常奇怪的0x40(%rsp,%r10,4)和0x60(%rsp,%r10,4)，经过观察我们发现这和我们象棋中蹩马脚的规则非常类似，即比如我们下一步的移动是(x-1,y+2),那么就会检测(x+0,y+1)处是否有墙，有墙就会被蹩马脚，即会检测我们新的坐标的位置在0x45f8(%rip)存的数组(以链表形式保存)中是否为0，如果为1的话则判定发生了撞墙，就会导致拆除失败。

```

1bd5: 48 8d 15 d4 45 00 00  lea     0x45d4(%rip),%rdx        # 61b0 <row0>
1bdc: 45 85 c0            test    %r8d,%r8d
1bdf: 7e 11               jle     1bf2 <func7+0x1fa>
1be1: b8 00 00 00 00      mov     $0x0,%eax
1be6: 48 8b 52 08         mov     0x8(%rdx),%rdx
1bea: 83 c0 01            add     $0x1,%eax
1bed: 41 39 c0            cmp     %eax,%r8d
1bf0: 75 f4               jne     1be6 <func7+0x1ee>
1bf2: 49 63 c3            movslq  %r11d,%rax
1bf5: b9 00 00 00 00      mov     $0x0,%ecx
1bfa: 80 3c 02 01         cmpb    $0x1,(%rdx,%rax,1)

```

这里就是检测我们的下一步(x+dx,y+dy)是否是墙了，是墙同样会导致拆除失败。

所以我们只需要打印出来地图，找到一条路线，既不会撞到墙也不会被墙蹩马脚，就可以了。通过查看地址，我们发现：

```

Row 0: 00 00 01 00 00 01 00 00
Row 1: 00 00 00 01 00 00 00 01
Row 2: 01 00 01 00 00 01 00 00
Row 3: 01 00 00 00 00 00 00 00
Row 4: 00 01 00 00 01 00 01 00
Row 5: 01 00 00 01 01 00 00 00
Row 6: 00 00 00 00 00 01 00 01
Row 7: 00 01 00 00 00 00 00 00

```

所以ccaac是一组可行的移动

反馈/收获/感悟/总结

这个lab还是很有趣的，但是我觉得可以适当给一些允许爆炸的次数.....就好比第一次炸不计入啥的，让大家都可以0成本看一次爆炸动画

参考的重要资料

1.ChatGPT:主要用于最开始询问gbd的用法，某些看不懂的汇编会贴他 <https://chatgpt.com/share/692a7f0f-d424-8007-abd0-30281aed3af2>